ANKARA ÜNİVERSİTESİ

MÜHENDİSLİK FAKÜLTESİ

BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

BLM4522

AĞ TABANLI PARALEL DAĞITIM SİSTEMLERİ

PROJE 2

VERİTABANI YEDEKLEME VE FELAKETTEN KURTARMA PLANI

Zeliha Kıyak

22290878

GitHub : https://github.com/zelihakiyak

VİDEO LİNKİ: https://youtu.be/OgD5-EoiihA

1. Giriş

Bu projede AdventureWorks2022 veritabanı üzerinde kapsamlı bir yedekleme ve felaketten kurtarma planı tasarlanmıştır. Proje kapsamında tam, diferansiyel ve transaction log yedeklemeleri gerçekleştirilmiştir. PowerShell ile otomatik yedekleme kurulmuş, felaket senaryosu simüle edilerek restore işlemi uygulanmış ve yedek doğrulama testleri yapılmıştır.

2. Yedekleme Stratejisi

Veritabanı yedeklemede üç temel yöntem kullanılmıştır. Her birinin farklı amacı ve kullanım senaryosu vardır.

Yedek Türü	Kapsam	Kullanım Amacı
Tam Yedek	Tüm veritabanı	Haftalık / ana yedek
Diferansiyel	Son tam yedekten bu yana değişen veriler	Günlük yedek
Transaction Log	Son log yedeğinden bu yana işlemler	Saatlik / point-in-time restore

3. Tam Yedek

Tam yedek veritabanının o anki eksiksiz kopyasını oluşturur. Tüm yedekleme stratejisinin temelidir. Diferansiyel ve log yedekleri bu yedeğe dayalı çalışır. FORMAT ve INIT parametreleri dosyayı sıfırdan yazmayı, STATS=10 ise her %10'da bir ilerlemeyi göstermeyi sağlar.

```
BACKUP DATABASE AdventureWorks2022
TO DISK = '...\Backup\AW_Full.bak'
WITH FORMAT,
     INIT,
     NAME = 'AdventureWorks2022 - Tam Yedek',
     STATS = 10;
```

Tam yedek tamamlandığında elde edilen sonuçlar aşağıdaki tabloda görülmektedir:

Metrik	Değer
İşlenen Sayfa	25.489 sayfa
Tamamlanma Süresi	0.263 saniye
Hız	757.134 MB/sn

4. Diferansiyel Yedek

Diferansiyel yedek son tam yedekten bu yana değişen verileri kaydeder. Tam yedeğe göre çok daha hızlı ve küçük boyutludur. Yedek öncesinde veritabanında değişiklik yapılarak diferansiyelin etkisi gözlemlenmiştir.

```
-- Önce test verisi eklendi
INSERT INTO Person.BusinessEntity (ModifiedDate, rowguid)
VALUES (GETDATE(), NEWID());

DECLARE @NewID INT = SCOPE_IDENTITY();

INSERT INTO Person.Person
(BusinessEntityID, PersonType, FirstName, LastName, ModifiedDate,
rowguid)
VALUES
(@NewID, 'EM', 'Test', 'Kullanici', GETDATE(), NEWID());

SELECT BusinessEntityID, FirstName, LastName
FROM Person.Person
WHERE FirstName = 'Test';

BACKUP DATABASE AdventureWorks2022
TO DISK = '...\Backup\AW_Diff.bak'
WITH DIFFERENTIAL, FORMAT, INIT,
     NAME = 'AdventureWorks2022 - Diferansiyel Yedek',
     STATS = 10;
```

Tam yedek ile diferansiyel yedek karşılaştırıldığında aşağıdaki sonuçlar elde edilmiştir:

Metrik	Tam Yedek	Diferansiyel	Fark
İşlenen Sayfa	25.489	393	%98.5 daha az
Süre	0.263 sn	0.017 sn	15x daha hızlı

Diferansiyelin 25.489 yerine sadece 393 sayfa işlemesi, yedekleme stratejisinin etkinliğini göstermektedir. Yalnızca eklenen test kaydı ve değişen sayfalar yedeklenmiştir.

5. Transaction Log Yedeği

Transaction log yedeği son log yedeğinden bu yana gerçekleşen tüm işlemleri kaydeder ve point-in-time restore yapabilmek için zorunludur. Transaction log yedeği alabilmek için veritabanının Recovery Modelinin FULL olması gerekmektedir.

```
-- Recovery Model FULL'e çevrildi
ALTER DATABASE AdventureWorks2022 SET RECOVERY FULL;

-- Recovery model değişikliği sonrası yeni tam yedek alındı
BACKUP DATABASE AdventureWorks2022
TO DISK = '...\Backup\AW_Full2.bak'
WITH FORMAT,
     INIT,
```

```
     NAME = 'Tam Yedek 2', STATS = 10;
INSERT INTO Person.BusinessEntity (ModifiedDate, rowguid)
VALUES (GETDATE(), NEWID());

DECLARE @NewID INT = SCOPE_IDENTITY();

INSERT INTO Person.Person
(BusinessEntityID, PersonType, FirstName, LastName, ModifiedDate,
rowguid)
VALUES (@NewID, 'EM', 'Log', 'Test1', GETDATE(), NEWID());

UPDATE Person.Person
SET FirstName = 'Guncellendi'
WHERE FirstName = 'Test' AND LastName = 'Kullanici';

-- Log yedeği alındı
BACKUP LOG AdventureWorks2022
TO DISK = '...\Backup\AW_Log1.bak'
WITH FORMAT, INIT,
     NAME = 'AdventureWorks2022 - Log Yedek 1',
     STATS = 10;
```

Log yedeği sadece 6 sayfa işledi. Bu beklenen bir sonuçtur; yalnızca log yedeği alınmadan önce yapılan 3 işlem (2 INSERT, 1 UPDATE) kaydedilmiştir.

6. Otomatik Yedekleme - PowerShell ve Task Scheduler

SQL Server Express Edition'da SQL Server Agent bulunmadığından Windows Task Scheduler ve PowerShell kombinasyonu kullanılmıştır. Aşağıdaki script her çalıştığında tarih damgalı yedek dosyası oluşturmakta ve sonucu log dosyasına yazmaktadır.

```
$ServerInstance = ".\SQLEXPRESS"
$Database = "AdventureWorks2022"
$Date = Get-Date -Format "yyyy-MM-dd_HH-mm"
$BackupFile = "$BackupPath\AW_Otomatik_$Date.bak"
$LogFile = "C:\Backup\yedek_log.txt"

try {
    Invoke-Sqlcmd -ServerInstance $ServerInstance -Query $Query
    $Mesaj = "$(Get-Date) - BASARILI: $BackupFile olusturuldu"
    Add-Content -Path $LogFile -Value $Mesaj
} catch {
    $Mesaj = "$(Get-Date) - HATA: $_"
    Add-Content -Path $LogFile -Value $Mesaj
}
```

Ardından Windows Task Scheduler ile her gece 02:00'de otomatik çalışacak görev oluşturuldu:

```
$Action = New-ScheduledTaskAction \
    -Execute "PowerShell.exe" \
    -Argument "-ExecutionPolicy Bypass -File
'C:\Backup\yedek_al.ps1'"

$Trigger = New-ScheduledTaskTrigger -Daily -At "02:00AM"

Register-ScheduledTask \
    -TaskName "AdventureWorks Otomatik Yedek" \
    -Action $Action -Trigger $Trigger -RunLevel Highest -Force
```

Görev doğrulandığında State: Ready sonucu alındı. Zamanlanmış görev başarıyla kuruldu.

7. Felaketten Kurtarma Senaryosu

Kaza ile veri silme senaryosu simüle edildi. Sales.SalesOrderHeader tablosunda 31.465 kayıt bulunduğu not alındıktan sonra tüm satış siparişleri silindi.

```
-- Felaket öncesi toplam kayıt
SELECT COUNT(*) AS ToplamKayit FROM Person.Person;
SELECT COUNT(*) AS ToplamKayit FROM Sales.SalesOrderHeader;

-- Felaket simülasyonu
DELETE FROM Sales.SalesOrderDetail;
DELETE FROM Sales.SalesOrderHeader;

-- Doğrulama: 0 satır geldi
SELECT COUNT(*) AS KalanKayit FROM Sales.SalesOrderHeader;
```

Restore işlemi Full2.bak → Log1.bak zinciri kullanılarak gerçekleştirildi. Bu zincir tutarlı olduğundan başarıyla tamamlandı.

```
-- Single User moduna alındı
ALTER DATABASE AdventureWorks2022
SET SINGLE_USER WITH ROLLBACK IMMEDIATE;

-- Tam yedek geri yüklendi
RESTORE DATABASE AdventureWorks2022
FROM DISK = '...\AW_Full2.bak'
WITH NORECOVERY, REPLACE, STATS = 10;

-- Log yedeği uygulandı ve veritabanı açıldı
RESTORE LOG AdventureWorks2022
FROM DISK = '...\AW_Log1.bak'
WITH RECOVERY, STATS = 10;
```

```
ALTER DATABASE AdventureWorks2022 SET MULTI_USER;

SELECT COUNT(*) AS ToplamKayit FROM Sales.SalesOrderHeader;
```

Restore işlemi sonucunda kayıt sayıları karşılaştırıldığında aşağıdaki sonuçlar elde edilmiştir:

Aşama	SalesOrderHeader Kayıt Sayısı	Durum
Felaket Öncesi	31.465 kayıt	Normal
Felaket Sonrası	0 kayıt	Felaket!
Restore Sonrası	31.465 kayıt	Tam Kurtarma

Tüm veriler eksiksiz geri getirildi. Restore işlemi başarıyla tamamlandı.

8. Yedek Doğrulama Testleri

RESTORE VERIFYONLY komutu kullanılarak tüm yedekler fiziksel olarak doğrulandı. Bu komut yedeği geri yüklemeden yalnızca bütünlüğünü kontrol eder.

```
RESTORE VERIFYONLY FROM DISK = '...\AW_Full.bak';

RESTORE VERIFYONLY FROM DISK = '...\AW_Diff.bak';

RESTORE VERIFYONLY FROM DISK = '...\AW_Log1.bak';

RESTORE VERIFYONLY FROM DISK = '...\AW_Otomatik_2026-04-16_14-
26.bak';
```

Yedek Dosyası	Sonuç
AW_Full.bak	The backup set on file 1 is valid
AW_Diff.bak	The backup set on file 1 is valid
AW_Log1.bak	The backup set on file 1 is valid
AW_Otomatik_2026-04-16_14-26.bak	The backup set on file 1 is valid

4 yedek dosyasının tamamı geçerli olarak doğrulandı.

9. Sonuç ve Değerlendirme

Bu projede AdventureWorks2022 veritabanı üzerinde kapsamlı bir yedekleme ve felaketten kurtarma planı başarıyla tasarlanmış ve uygulanmıştır.

9.1 Yedekleme Stratejisi

Üç farklı yedekleme yöntemi uygulanmıştır. Tam yedek ile veritabanının eksiksiz bir kopyası alınmıştır. 25.489 sayfa 0.263 saniyede 757 MB/sn hızıyla yedeklenmiştir. Diferansiyel yedek, tam yedekten sonra yapılan değişiklikleri kapsayarak yalnızca 393 sayfa işlemiştir. Bu, tam yedeğe kıyasla %98.5 daha az veri işlenmesi anlamına gelir ve yedekleme süresini 15 kat kısaltmıştır. Transaction log yedeği ise yalnızca 6 sayfa işleyerek son işlemleri kayıt altına almıştır. Bu üç yöntemin birlikte kullanılması, hem depolama alanını verimli kullanmayı hem de veri kaybını en aza indirmeyi sağlamaktadır.

9.2 Otomatik Yedekleme

SQL Server Express Edition'da SQL Server Agent bulunmadığından PowerShell scripti ve Windows Task Scheduler kombinasyonu kullanılmıştır. Script her çalıştığında tarih damgalı yedek dosyası oluşturmakta ve sonucu log dosyasına yazmaktadır. Görev her gece 02:00'de otomatik çalışacak şekilde yapılandırılmış olup State: Ready durumu ile doğrulanmıştır. Bu yöntem production ortamlarında da yaygın olarak kullanılmakta olup SQL Server Agent'a işlevsel bir alternatif sunmaktadır.

9.3 Felaketten Kurtarma

Kaza ile veri silme senaryosu gerçekçi biçimde simüle edilmiştir. Sales.SalesOrderHeader tablosundaki 31.465 kaydın tamamı silinerek felaket ortamı oluşturulmuştur. Ardından Full2.bak ve Log1.bak yedekleri kullanılarak restore işlemi gerçekleştirilmiştir. NORECOVERY modu ile tam yedek uygulanmış, ardından RECOVERY modu ile log yedeği uygulanarak veritabanı açılmıştır. Restore sonrası yapılan doğrulama sorgusunda 31.465 kaydın tamamının eksiksiz geri geldiği görülmüştür. Bu sonuç, doğru bir yedekleme stratejisi ile veri kaybı yaşanmadan tam kurtarma yapılabildiğini kanıtlamaktadır.

9.4 Yedek Doğrulama

RESTORE VERIFYONLY komutu ile alınan 4 yedek dosyasının tamamı fiziksel olarak doğrulanmıştır. Tam yedek, diferansiyel yedek, log yedeği ve otomatik yedek dosyalarının tamamı için "The backup set on file 1 is valid" sonucu alınmıştır. Yedek doğrulama, production ortamlarında kritik bir adımdır. Alınan yedeğin gerçekten geri yüklenebilir olduğunu garantiler.